

Hydraulic System Condition Monitoring

MLOps Pipeline — From Training to Production

Personne A Personne B Personne C Personne D



DATA713 — Mars 2026

Context & Problem

Use case: Industrial predictive maintenance

Dataset: UCI — Condition Monitoring of Hydraulic Systems

- 2205 operating cycles $\times$ 17 sensors
- 4 component conditions to predict simultaneously
- Labels from `profile.txt` (supervised)

Formulation: Multi-class multi-output classification

Not anomaly detection — labeled data allows per-component diagnostics

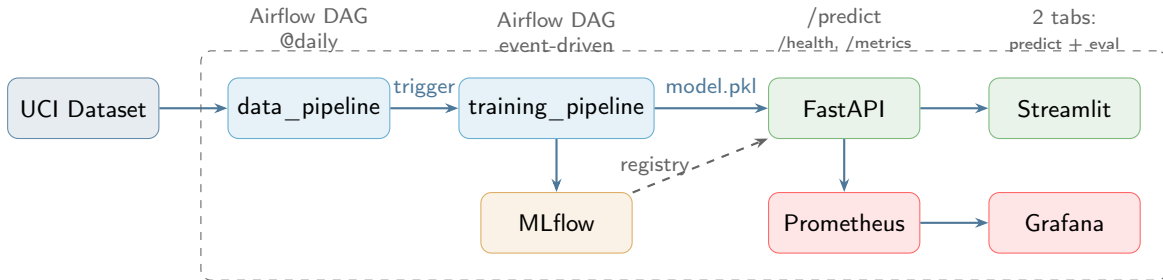
Target	Component	Classes
cooler	Cooler	3
valve	Valve	4
pump	Pump	3
accumulator	Accum.	4

17 sensors:

PS1–PS6, EPS1, FS1–FS2,
TS1–TS4, VS1, CE, CP, SE

Feature eng.: mean per cycle

Architecture Overview



Dev: docker compose up (9 services)

Prod: K8s manifests ready, CD conditional

ML Pipeline & Continuous Training

Model: MultiOutputClassifier(RF)

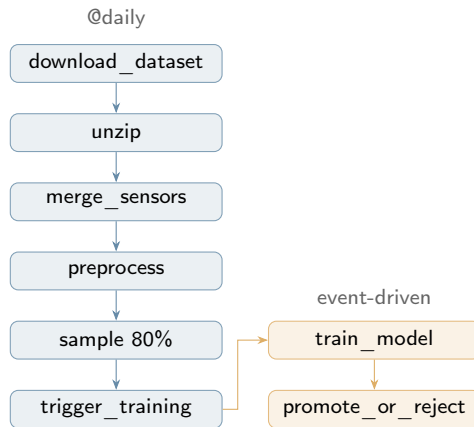
Results (test set 20%):

Target	Acc.	F1 _{macro}	F1 _{weighted}
Cooler	1.000	1.000	1.000
Valve	0.948	0.947	0.948
Pump	0.993	0.993	0.993
Accumulator	0.990	0.990	0.990

MLflow tracks:

- Params, metrics per target, model artifact
- JSON report (confusion matrices)
- Model Registry: promote / reject / archive

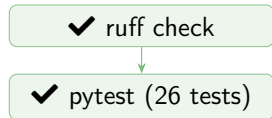
CT Logic (2 DAGs):



Why 80% sampling? Static dataset — sampling simulates data variability so champion/challenger

CI/CD Pipeline

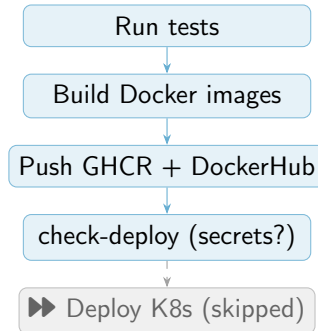
CI (every push & PR)



26 tests in 3 suites:

Suite	Tests
DAG structure	9
Model training	7
Preprocessing	10

CD (push to main)



Tags: latest + git SHA (rollback)

K8s deploy conditional on KUBECONFIG secret

FastAPI

- **POST /predict**
Pydantic body, 17 sensors → 4 states
- **GET /health**
Readiness probe
- **GET /metrics**
Prometheus instrumentation
- **GET /docs**
Swagger auto-generated

Streamlit

Tab 1 — Prediction

- 17 sensor inputs
- Calls API → 4 predicted states

Tab 2 — Model Evaluation

- Accuracy, Precision, Recall, F1
- Confusion matrices
- Classification reports

Monitoring

- **Prometheus**
Scrapes API /metrics
+ Node Exporter
- **Grafana**
Dashboard auto-provisioned
on docker compose up
- **Alerting**
Airflow email callbacks
on DAG failure

 Screenshots of each component available — see backup slides

Team organization:

	Scope
A	Data & ML pipeline, MLflow integration
B	Airflow DAGs, CT, CI/CD, 26 tests, docs
C	FastAPI API, Streamlit webapp
D	Docker Compose, K8s, CD, monitoring

Git workflow:

- Feature branches → PR → main
- CI gates (ruff + pytest)
- 20+ PRs, conventional commits

Honest limitations:

- Dataset is static — CT designed for production but trained on sampled data
- No K8s cluster — CD pipeline ready, deploy step skipped
- API loads `model.pkl` locally, not from MLflow Registry
- Heavy Airflow imports — learned to defer inside task functions

Key learning:

Separation of concerns — DAGs orchestrate, `src/` owns business logic, API serves, webapp displays.

Objectives Coverage

10/10 required objectives:

#	Objective	Status
1	Data extraction & preprocessing	✓
2	ML model	✓
3	Model registry (MLflow)	✓
4	Airflow retraining pipeline	✓
5	MLflow experiment tracking	✓
6	API (FastAPI + Swagger)	✓
7	WebApp (Streamlit, 2 tabs)	✓
8	Continuous Training (CT)	✓
9	Docker + K8s + CI/CD	✓
10	GitHub versioning & docs	✓

Bonus realized (3/8):

#	Bonus	Status
12	Monitoring (Prom+Grafana)	✓
14	Model versioning/rollback	✓
17	Email alerting	✓
11	Live data	✗
13	Load testing (Locust)	✗
15	Human in the loop	✗
16	API access mgmt	✗

Stack:

Python 3.11 · scikit-learn · MLflow
Airflow · FastAPI · Streamlit
Docker · K8s · GitHub Actions
Prometheus · Grafana · uv

Screenshots to be added after
docker compose up validation

- Airflow UI (both DAGs, task graph)
- MLflow UI (experiment runs, metrics)
- Swagger /docs (Pydantic schema)
- Streamlit Tab 1 (prediction) + Tab 2 (evaluation)
- Grafana dashboard
- GitHub Actions (CI green, CD pipeline)
- GitHub PRs with reviews